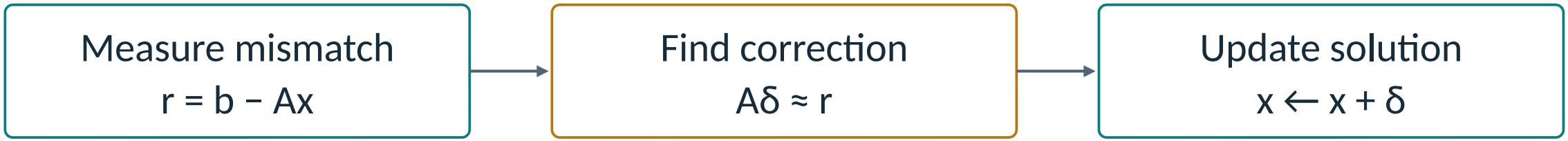
# A digital iterative solver with photonic matrix–vector products

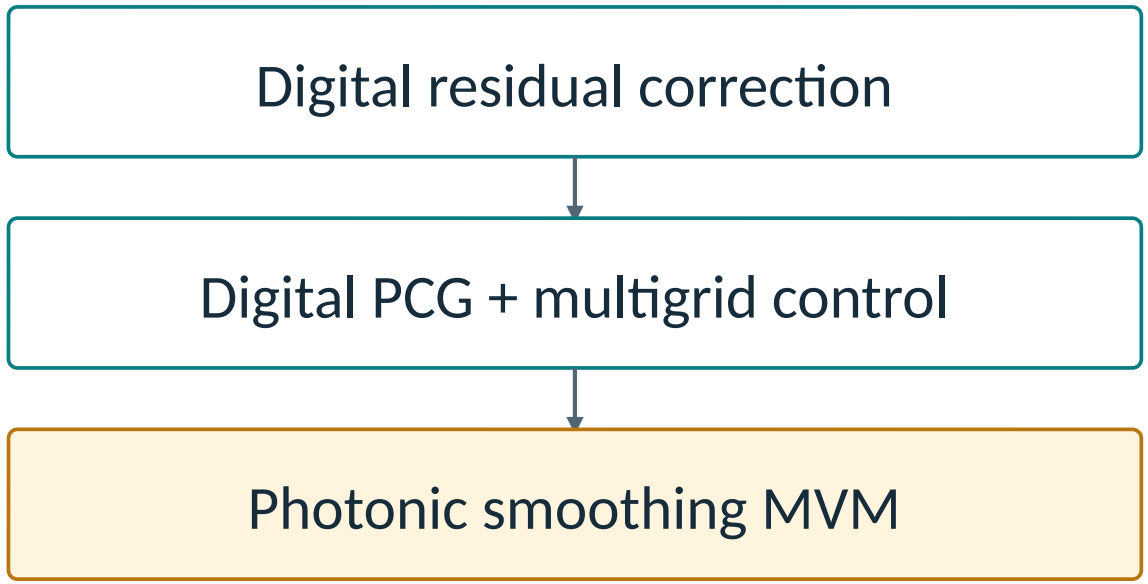
Fixed matrices and changing correction vectors solve Ax = b




Project implementation inspired by Lee et al. (2025)

# The paper assigns smoothing products to optics

## Numerical structure adopted here




## Current project

Noisy 2D emulation

+

ideal 1D demonstrator

Hardware diagrams show functional assignments.

Research question: can approximate smoothing reduce digital work while reliable residuals retain accuracy?

Lee et al., arXiv:2511.01005, pp. 3–6, 15–16

# Three implementations, different settings

| | Lee Poisson experiment | main.py | Browser demonstrator |
|---|---|---|---|
| Role | Research foundation | Noisy numerical evidence | Explanatory model |
| Problem | 2D capacitor | 64 × 64 = 4096 unknowns | 1D, 64 unknowns |
| AMG levels | 2 | 4096 → 704 in this run | 64 → 22 → 8 → 3 |
| Hybrid sweeps | 4 pre / 0 post | 4 pre / 0 post | 3 pre / 3 post |
| MVM behavior | Physical measurements | Input quantization + noise | Ideal floating point |

Main explanation: browser’s 1D model. Research evidence: main.py’s separate 2D run.

Lee et al.; main.py; web/index.html; smoother.py

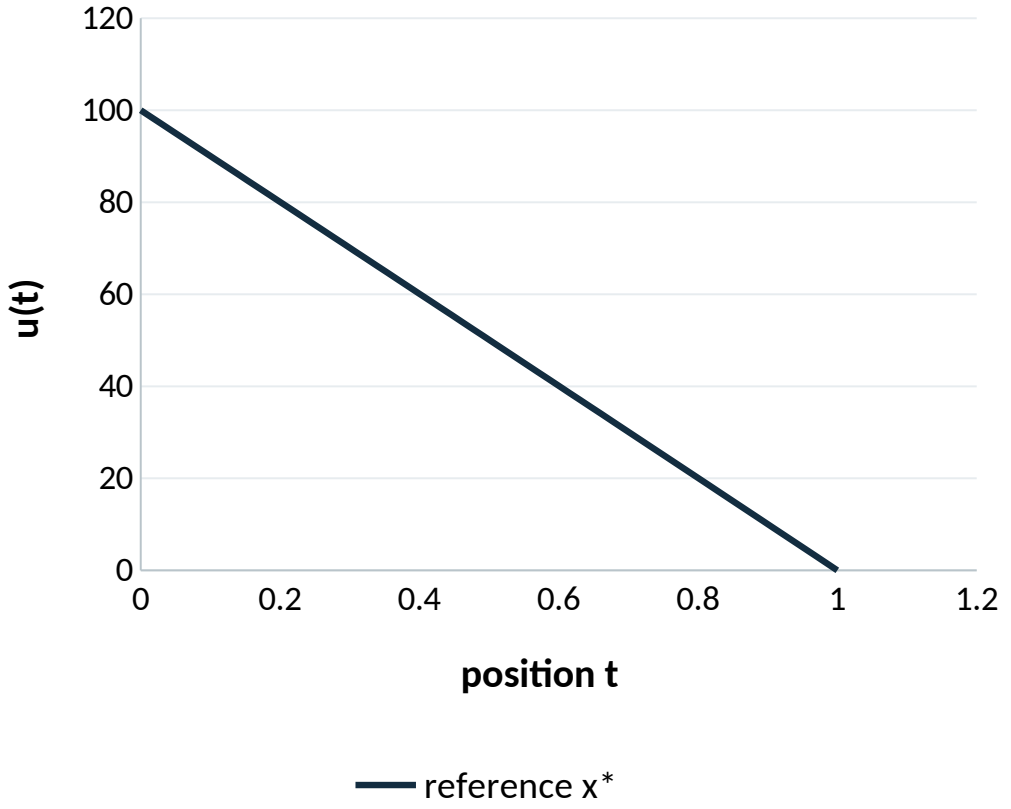
# The unknown vector stores sampled solution values

Ideal 1D teaching model

$$-u''(t) = 1$$
$$u(0) = 100, \quad u(1) = 0$$

$$(2x_i - x_{i-1} - x_{i+1}) / h^2 = 1$$
$$h = 1 / 65, \quad x \in \mathbb{R}^{64}$$

$$A_0 = h^{-2}\,\mathrm{tridiag}(-1, 2, -1)$$
$$b = [422501, 1, \ldots, 1]^T$$




$x = [u(t_1), \ldots, u(t_{64})]^T$. Iteration improves an equilibrium estimate; it is not physical time.

# The residual becomes the next correction problem

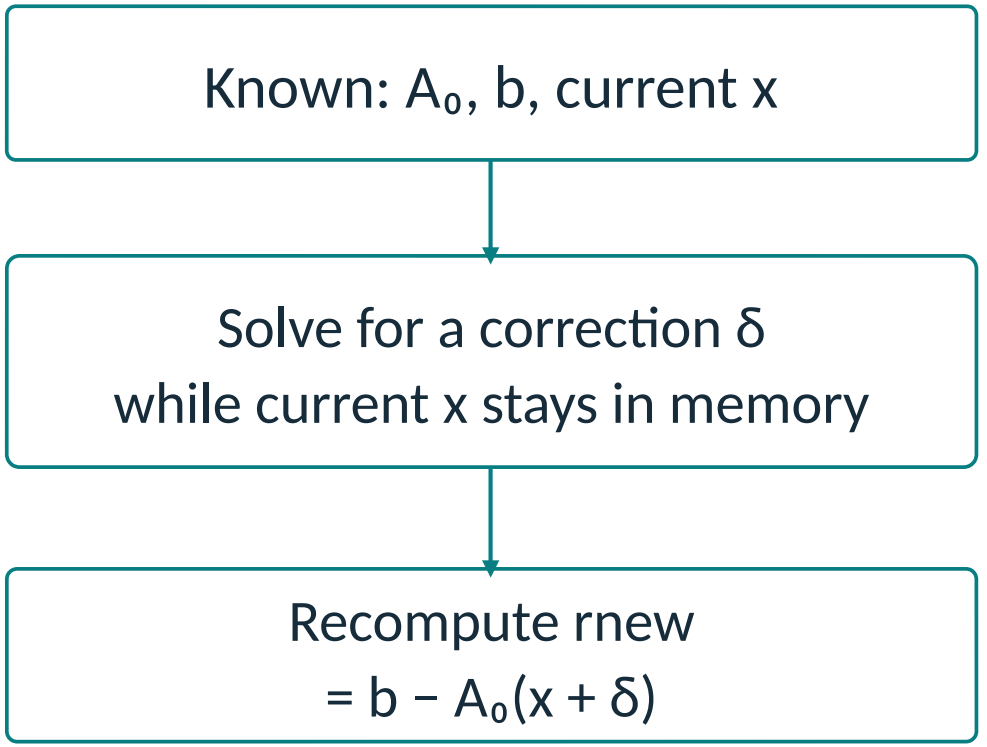


$$r = b - A_0 x$$

$$A_0(x^* - x) = r$$

Find $\delta$ with $A_0\delta \approx r$

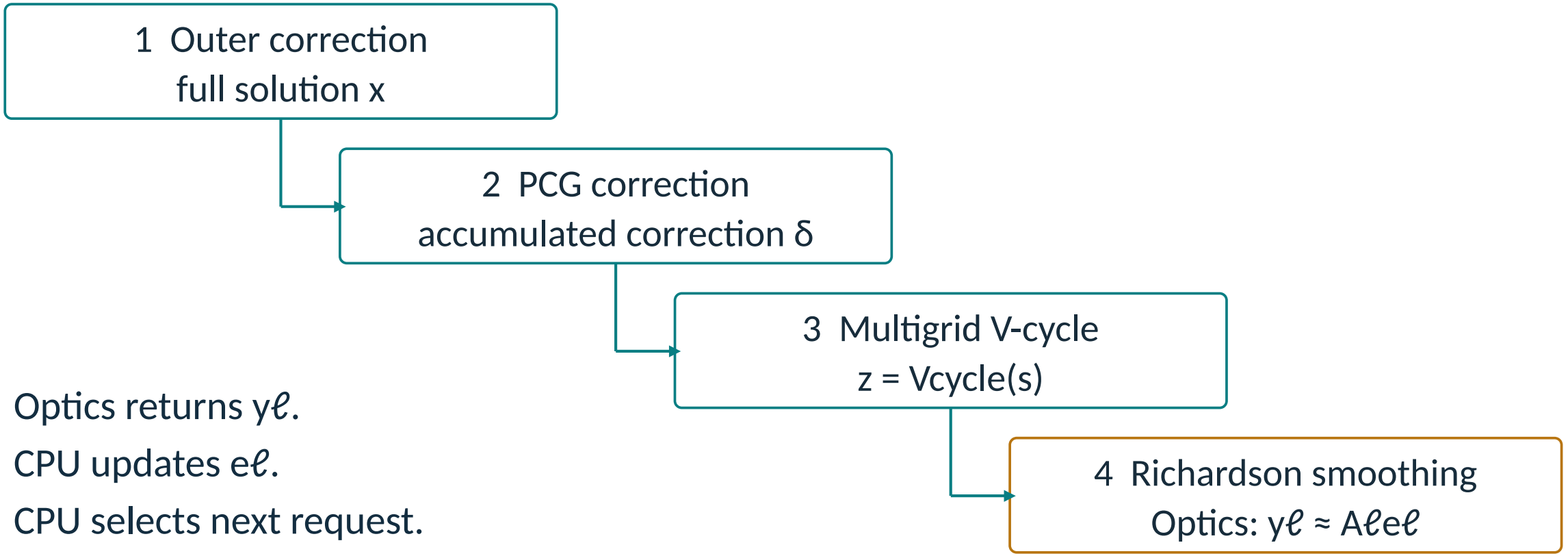
Then $x_{new} = x + \delta$




The exact solution $x^*$ is only a reference for evaluation. The algorithm uses the computable residual.

# One solver map connects all four routines

Read downward for calls; read upward for returned values



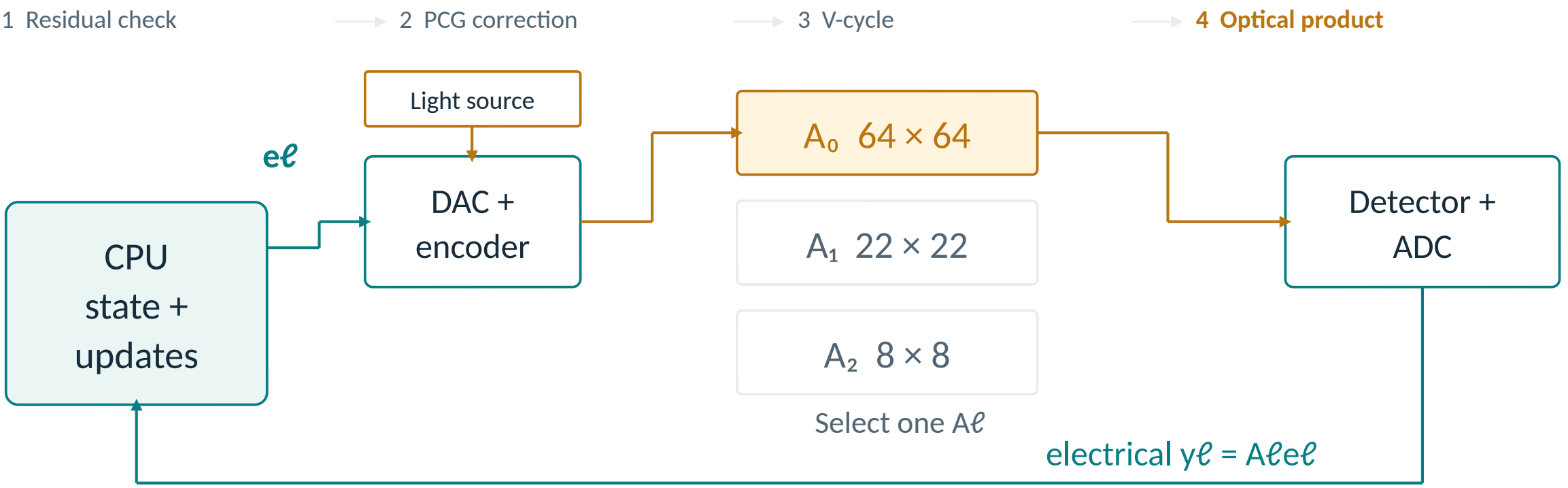

Optics returns $y_\ell$.

CPU updates $e_\ell$.

CPU selects next request.

Changing vectors: $x \to \delta \to s \to e_\ell$. Fixed operators: $A_\ell$, $P_\ell$, $R_\ell$.

# The CPU closes every optical smoothing loop




## CPU update: $e_\ell \leftarrow e_\ell + \omega_\ell (g_\ell - y_\ell)$

Amber: optical product. Teal: electrical data and CPU work. The full solution x is not the encoded vector.

web/index.html: PhotonicSolver numerical core and embedded sparse matrix data; web/README.md hardware mapping

# A real smoothing request changes the local vector

1 Residual check → 2 PCG correction → 3 V-cycle → **4 Optical product**

| Stored or transferred quantity | Actual first three entries |
|---|---|
| $g_0$, retained on CPU | $[4.225e+05, 1, 1, \ldots]^T$ |
| $e_0$, encoded input | $[25.01, 5.921e-05, 5.921e-05, \ldots]^T$ |
| $y_0 = A_0 e_0$, detected product | $[2.114e+05, -1.057e+05, 0, \ldots]^T$ |
| new $e_0 = e_0 + \omega_0(g_0 - y_0)$ | $[37.51, 6.257, 0.0001184, \ldots]^T$ |

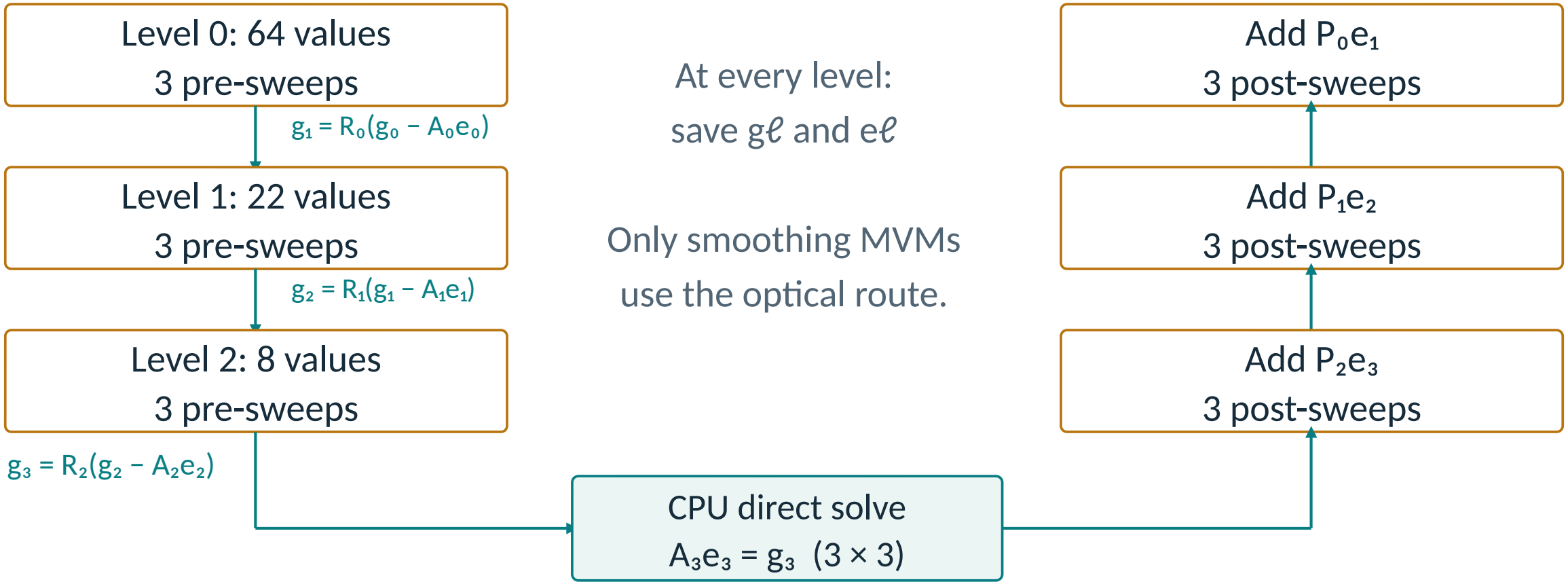
$\omega_0 = 5.9206167e-05$ · local vectors have 64 entries · full x is unchanged

Next sweep encodes the updated $e_0$ through the same fixed matrix $A_0$.

web/index.html: PhotonicSolver numerical core and embedded sparse matrix data; zero-start event trace

# The multigrid V-cycle

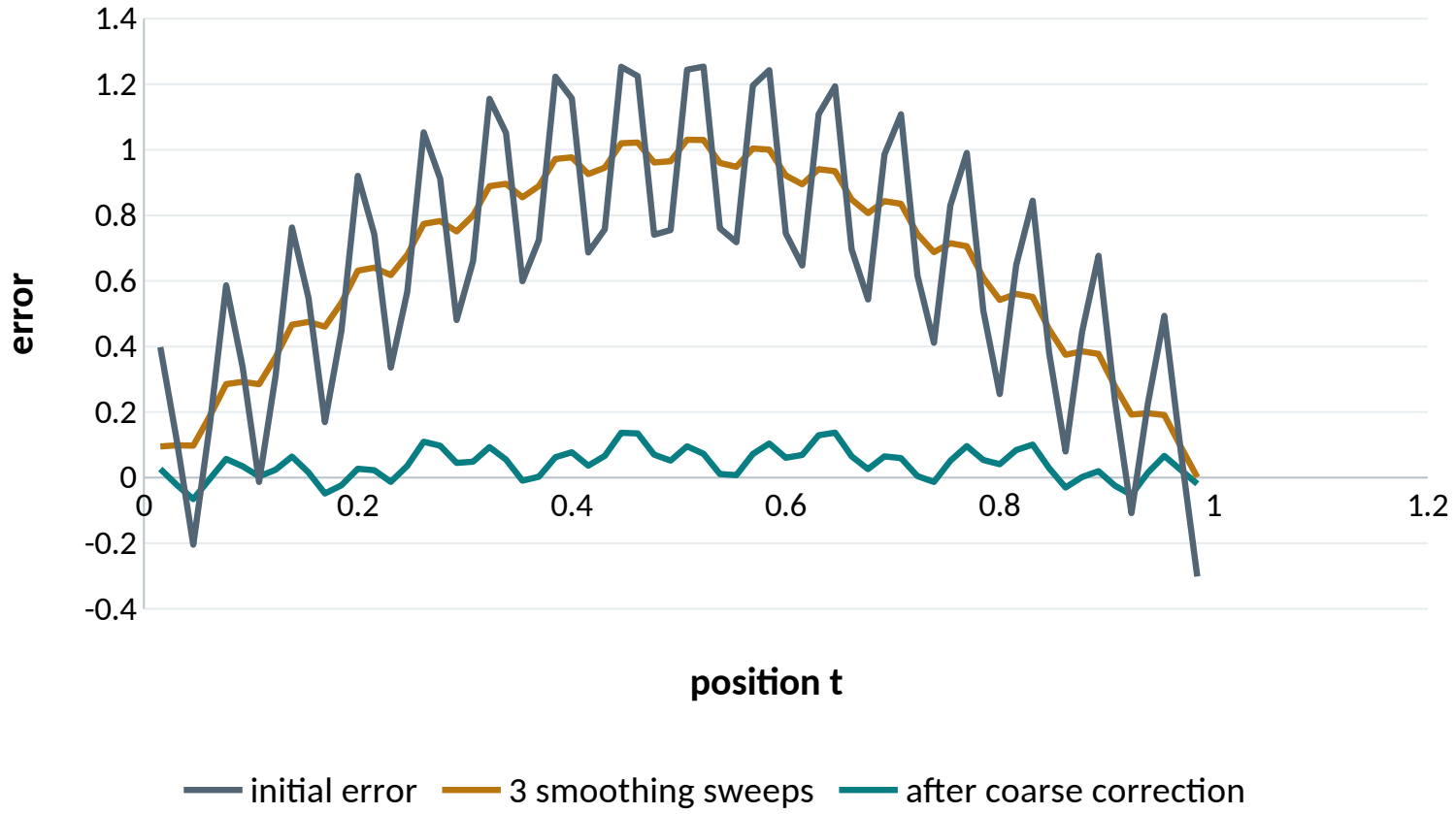
1 Residual check → 2 PCG correction → **3 V-cycle** → 4 Optical product




Return $z = e_0$ to PCG. The V-cycle supplies an approximate correction, not the final PDE solution.

# Smoothing and coarse correction

1 Residual check → 2 PCG correction → **3 V-cycle** → 4 Optical product




$$\varepsilon_{new} = (I - \omega A)\varepsilon$$

Mode j shrinks by $1 - \omega\lambda_j$

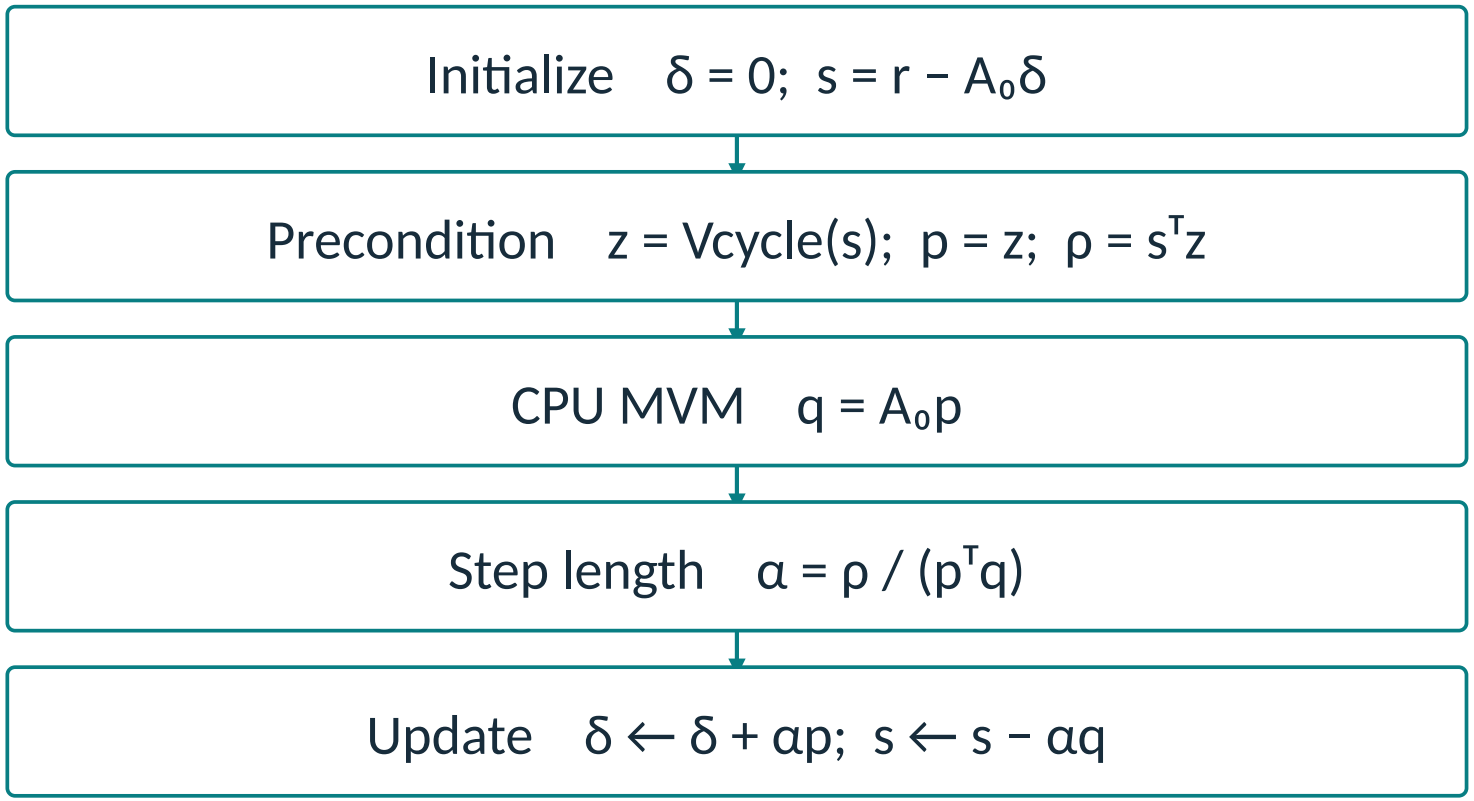
Error norm

$6.04 \rightarrow 5.70 \rightarrow 0.50$

Three post-sweeps reduce the norm further to 0.422. The initial error is prescribed for this test.

# PCG turns a V-cycle correction into a search step

1 Residual check → **2 PCG correction** → 3 V-cycle → 4 Optical product




p: direction

α: step length

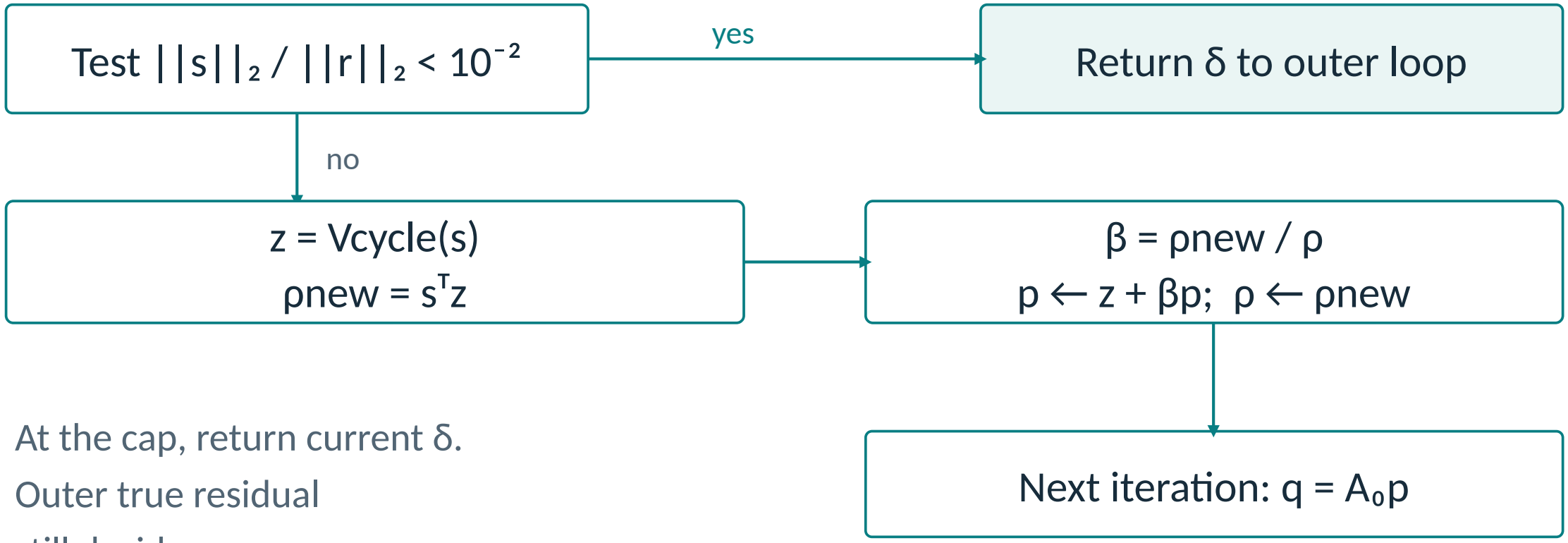
δ: correction so far

PCG arithmetic: CPU Vcycle invokes optical products.

The original full estimate x stays fixed until this correction solve returns δ.

# PCG stopping and direction updates

1 Residual check → **2 PCG correction** → 3 V-cycle → 4 Optical product




At the cap, return current δ.

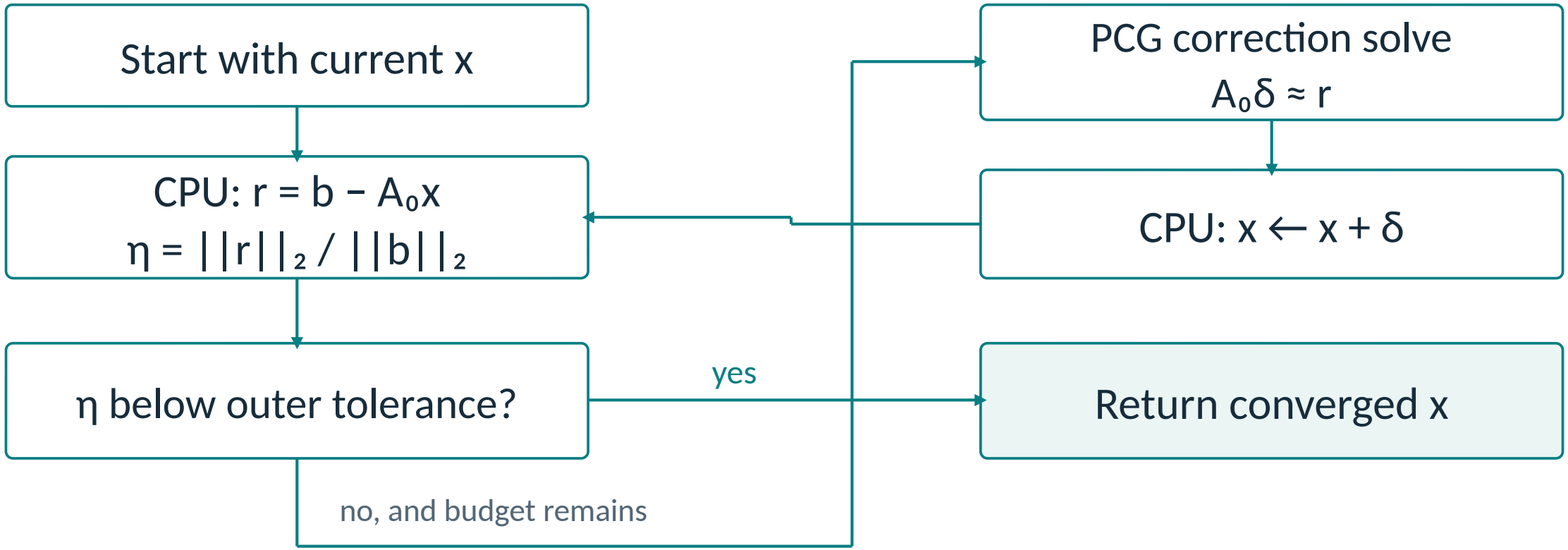
Outer true residual

still decides success.

$z = Vcycle(s)$ is an action approximating $A_0^{-1}s$. No inverse matrix is explicitly stored.

# Outer correction and true-residual checks

1 Residual check → 2 PCG correction → 3 V-cycle → 4 Optical product




Limit or numerical error: return an unverified approximation with its status, never label it converged.

web/index.html: PhotonicSolver numerical core and embedded sparse matrix data; main.py solve_with_restarts

# The hierarchy is fixed while its vectors change

Browser operator inventory: 4 A matrices + 3 P matrices + 3 R matrices

| Level | Operator and nonzeros | Prolongation $P\ell$ | Restriction $R\ell$ | Assignment |
|---|---|---|---|---|
| 0 | $A_0$: 64 × 64, 190 | 64 × 22 | 22 × 64 | A: optics + CPU |
| 1 | $A_1$: 22 × 22, 64 | 22 × 8 | 8 × 22 | A: optics + CPU |
| 2 | $A_2$: 8 × 8, 22 | 8 × 3 | 3 × 8 | A: optics + CPU |
| 3 | $A_3$: 3 × 3, 7 | — | — | CPU direct solve |

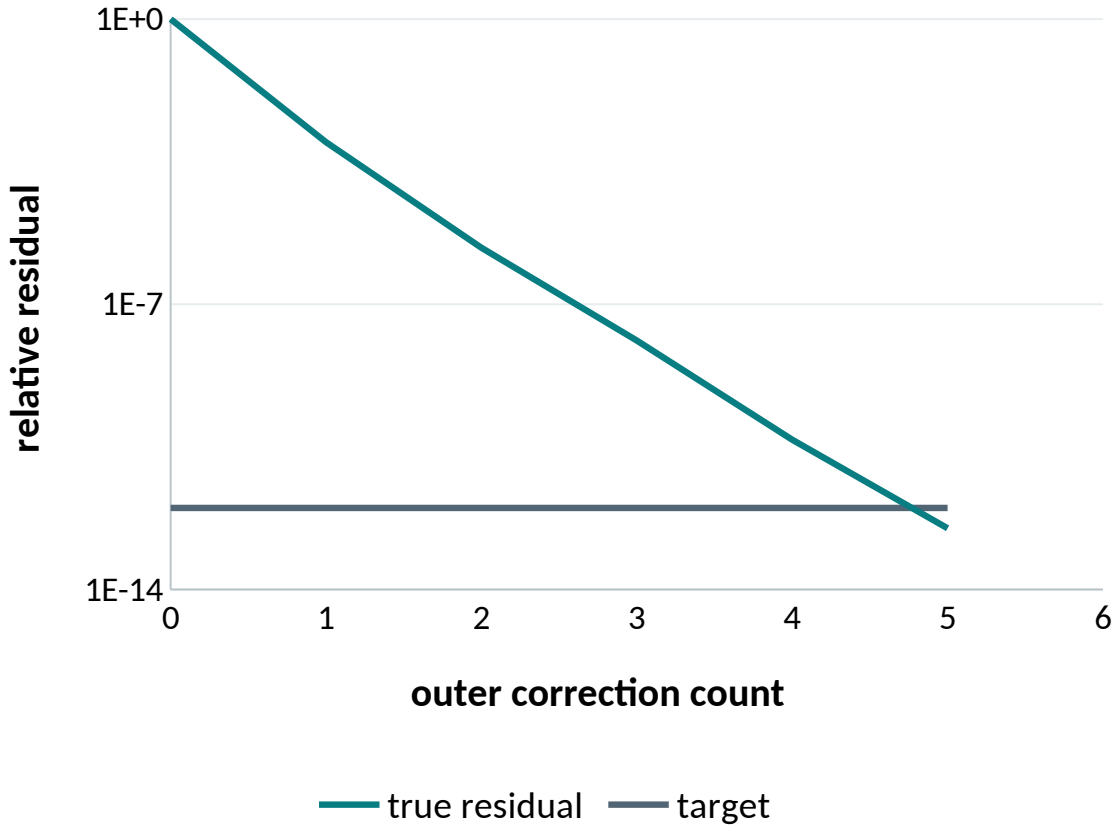
$$A\ell+1 = R\ell\, A\ell\, P\ell \qquad R\ell = P\ell^{T} \qquad \omega\ell \approx 1 / \lambda\max(A\ell)$$

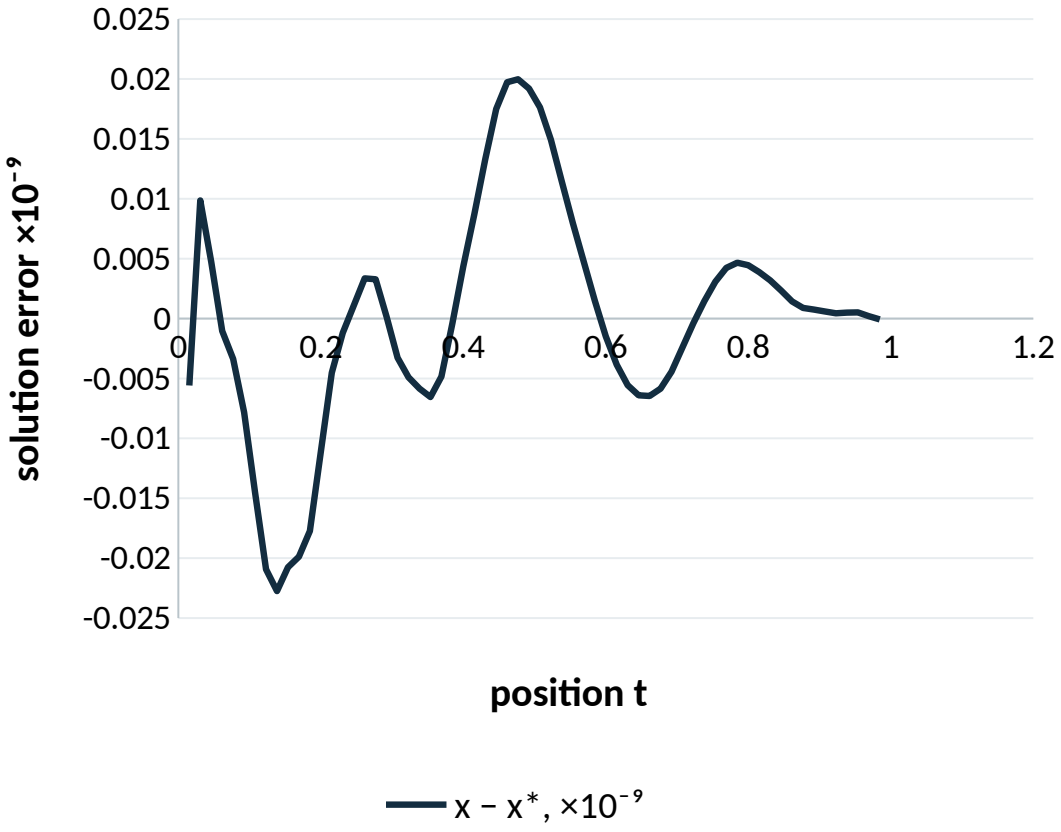
The CPU retains separate local vectors at each level. Coarse vectors are coefficients in algebraic spaces.

web/index.html: PhotonicSolver numerical core and embedded sparse matrix data; Galerkin and transpose identities verified

# Convergence of the ideal browser model

Ideal 1D model. Zero starting guess. Target $10^{-12}$.




Final $\eta = 3.17 \times 10^{-13}$ 180 optical calls 111 digital calls 10 V-cycles

web/index.html: PhotonicSolver numerical core and embedded sparse matrix data; fresh zero-start execution

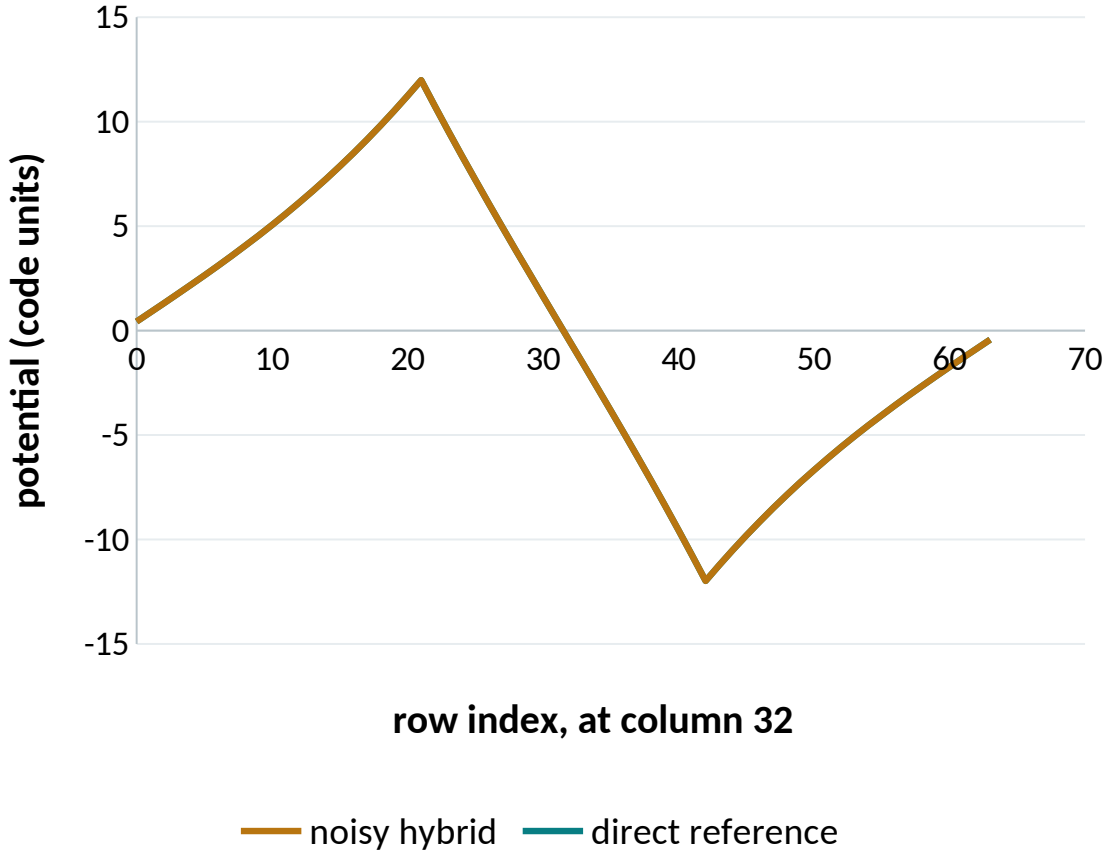
# The separate 2D noisy experiment

main.py: 4096 unknowns, two AMG levels, input quantization and output noise

$$4x_{ij} - x_{i-1,j} - x_{i+1,j} - x_{i,j-1} - x_{i,j+1} = b_{ij}$$




64 × 64 interior grid; exterior x = 0




Verified relative residual: $3.96 \times 10^{-11}$. Relative solution error: 6.95e-13.

main.py: original solver, observed by presentation/build/collect_python.py

# Counted digital products in the 2D run

Same 2D problem and final relative-residual target $10^{-10}$. Counter scope matters.

| Method | Digital A calls | Analog A calls | True relative residual |
|---|---|---|---|
| Plain CG | 148 | 0 | $7.8 \times 10^{-11}$ |
| Digital MG–CG | 60 | 0 | $6.1 \times 10^{-11}$ |
| Noisy hybrid + restarts | 45 | 60 | $4.0 \times 10^{-11}$ |

**148 / 45 = 3.29**

Ratio of counted digital products

No measured hardware speedup

Transfers and repeated 704 × 704 coarse solves are excluded. Browser counts use a different convention.

main.py: original solver, observed by presentation/build/collect_python.py; one controlled run, setup seed 0

# Implemented behavior and open questions

| Implemented and checked | Still to establish |
| --- | --- |
| Nested residual / CG / multigrid computation | Variable-preconditioner convergence across trials |
| Input quantization and additive noise in main.py | Calibration against measured optical errors |
| Fixed matrix blocks and CPU feedback in demonstrator | Signed encoding, sparse tiling, and physical routing |
| True residuals and algorithmic counters | End-to-end latency and energy, including transfers |

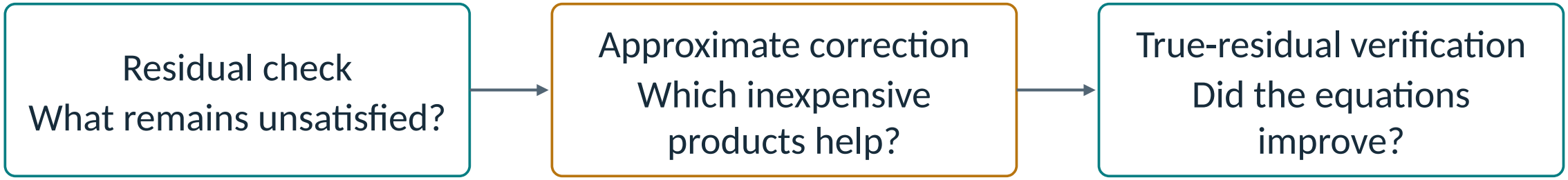
Related work: Zhu et al. (2024) provides a coherent signed-MVM architecture with digital memory and control.

main.py: original solver, observed by presentation/build/collect_python.py; web/README.md; Lee Methods

# Next validation: full computational cost

**Optics computes matrix–vector products.**

**The CPU builds and verifies the solution.**




Next: calibrated noise + repeated trials + full cost accounting, with the same sparse operators.

main.py: original solver, observed by presentation/build/collect_python.py; web/README.md

# Complete recurrence and termination conditions

Technical appendix: browser configuration

| Routine | Ordered operations |
| --- | --- |
| Outer initialization | Choose x. Load A$\ell$, P$\ell$, R$\ell$, $\omega\ell$, b. |
| Outer check | $r = b - A_0 x$; test $\|\|r\|\| / \|\|b\|\| < 10^{-12}$. |
| PCG initialization | $\delta = 0$; $s = r - A_0\delta$; $z = V(s)$; $p = z$; $\rho = s^T z$. |
| PCG step | $q = A_0 p$; $\alpha = \rho/(p^T q)$; $\delta \mathrel{+}= \alpha p$; $s \mathrel{-}= \alpha q$. |
| PCG check | Return $\delta$ if $\|\|s\|\| / \|\|r\|\| < 10^{-2}$. |
| New direction | $z = V(s)$; $\rho new = s^T z$; $\beta = \rho new/\rho$; $p = z + \beta p$; $\rho = \rho new$. |
| Caps / return | PCG: 20 steps. Outer: 10 corrections, then final true-residual check. |
| Outer update | $x \mathrel{+}= \delta$; return to fresh outer residual. |

# Recursive V-cycle and smoothing pseudocode

Every operation belongs to a CPU or optical assignment

```
V(l, g):
  if l = 3: return solve(A₃, g)
  e = 0
  repeat 3 times: e = Smooth(l, e, g)
  d = g − Aℓe                          [CPU]
  gc = Rℓd                             [CPU]
  ec = V(l + 1, gc)
  e = e + Pℓec                         [CPU]
  repeat 3 times: e = Smooth(l, e, g)
  return e
```

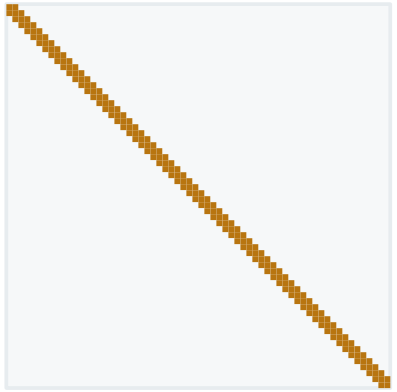
```
Smooth(l, e, g):

encode e
y = Aℓe      [optical]
detect y
e = e + ωℓ(g − y)
                [CPU]
return e
```

At level 0, $g_0 = s$ and V(0, s) returns z. The outer solution x is separate.

# Fine operators $A_0$ and $A_1$

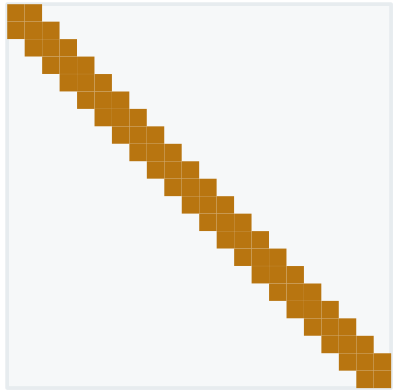
$A_0$: 64 × 64, 190 nonzeros. $A_1$: 22 × 22, 64 nonzeros.



$A_0$ fixed

CPU residuals

Optical smoothing



$A_1$ fixed

CPU residuals

Optical smoothing

| i / j | 1 | 2 | 3 | 4 |
|---|---|---|---|---|
| 1 | 8450.000 | -4225.000 | 0.000 | 0.000 |
| 2 | -4225.000 | 8450.000 | -4225.000 | 0.000 |
| 3 | 0.000 | -4225.000 | 8450.000 | -4225.000 |
| 4 | 0.000 | 0.000 | -4225.000 | 8450.000 |

Upper-left 4 × 4 excerpt; rounded to 3 decimals

| i / j | 1 | 2 | 3 | 4 |
|---|---|---|---|---|
| 1 | 1313.952 | -401.563 | 0.000 | 0.000 |
| 2 | -401.563 | 938.692 | -466.913 | 0.000 |
| 3 | 0.000 | -466.913 | 938.836 | -469.236 |
| 4 | 0.000 | 0.000 | -469.236 | 938.884 |

Upper-left 4 × 4 excerpt; rounded to 3 decimals

# Coarse operators $A_2$ and $A_3$

Full matrices shown. Entries rounded to 3 decimals.

| i / j | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 |
|---|---|---|---|---|---|---|---|---|
| 1 | 151.590 | -25.182 | 0.000 | 0.000 | 0.000 | 0.000 | 0.000 | 0.000 |
| 2 | -25.182 | 104.692 | -52.098 | 0.000 | 0.000 | 0.000 | 0.000 | 0.000 |
| 3 | 0.000 | -52.098 | 104.328 | -52.164 | 0.000 | 0.000 | 0.000 | 0.000 |
| 4 | 0.000 | 0.000 | -52.164 | 104.328 | -52.164 | 0.000 | 0.000 | 0.000 |
| 5 | 0.000 | 0.000 | 0.000 | -52.164 | 104.328 | -52.164 | 0.000 | 0.000 |
| 6 | 0.000 | 0.000 | 0.000 | 0.000 | -52.164 | 104.328 | -52.111 | 0.000 |
| 7 | 0.000 | 0.000 | 0.000 | 0.000 | 0.000 | -52.111 | 104.642 | -25.125 |
| 8 | 0.000 | 0.000 | 0.000 | 0.000 | 0.000 | 0.000 | -25.125 | 153.967 |

$A_2$ (8 × 8): optical smoothing + digital residuals

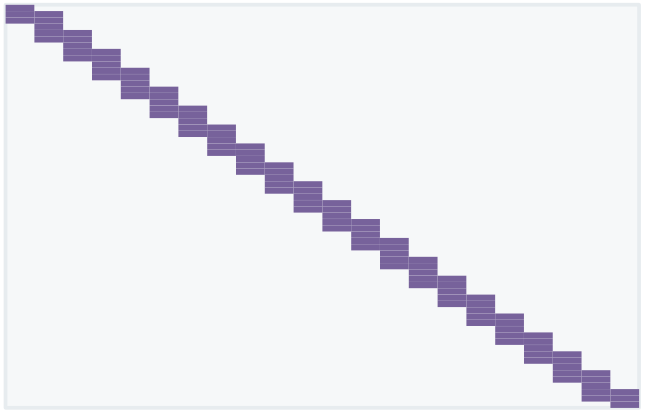
| i / j | 1 | 2 | 3 |
|---|---|---|---|
| 1 | 15.416 | -1.094 | 0.000 |
| 2 | -1.094 | 11.671 | -4.847 |
| 3 | 0.000 | -4.847 | 17.878 |

$A_3$ (3 × 3)

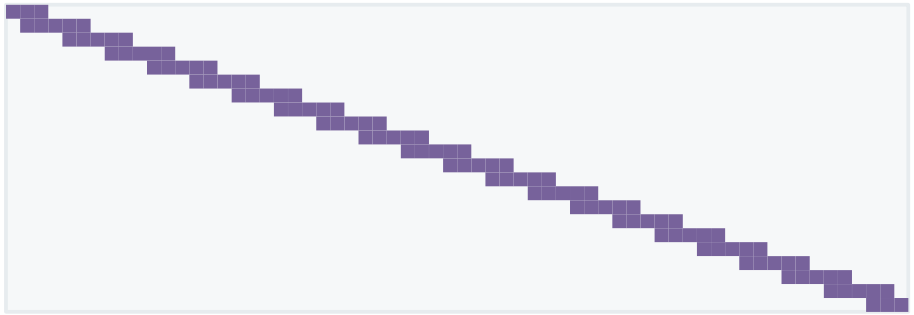
CPU direct solve

# Transfer operators $P_0$ and $R_0$

$P_0$: 64 × 22. $R_0$: 22 × 64. Fixed algebraic transfer operators.



coarse correction to fine



fine residual to coarse: gnext = R d

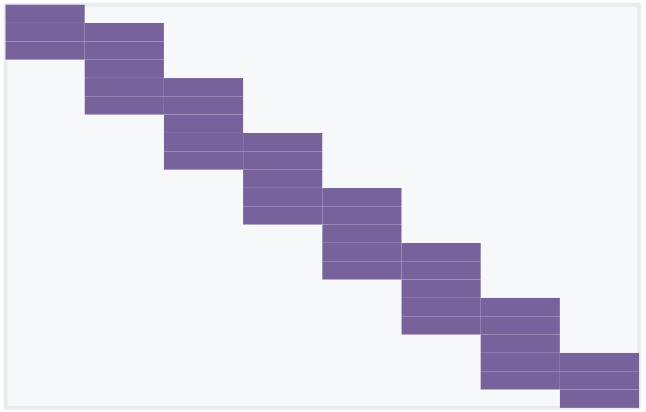
| i / j | 1 | 2 | 3 | 4 |
|---|---|---|---|---|
| 1 | 0.447 | 0.000 | 0.000 | 0.000 |
| 2 | 0.446 | 0.156 | 0.000 | 0.000 |
| 3 | 0.299 | 0.349 | 0.000 | 0.000 |
| 4 | 0.000 | 0.572 | 0.000 | 0.000 |
| 5 | 0.000 | 0.415 | 0.183 | 0.000 |

| i / j | 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|---|
| 1 | 0.447 | 0.446 | 0.299 | 0.000 | 0.000 |
| 2 | 0.000 | 0.156 | 0.349 | 0.572 | 0.415 |
| 3 | 0.000 | 0.000 | 0.000 | 0.000 | 0.183 |
| 4 | 0.000 | 0.000 | 0.000 | 0.000 | 0.000 |

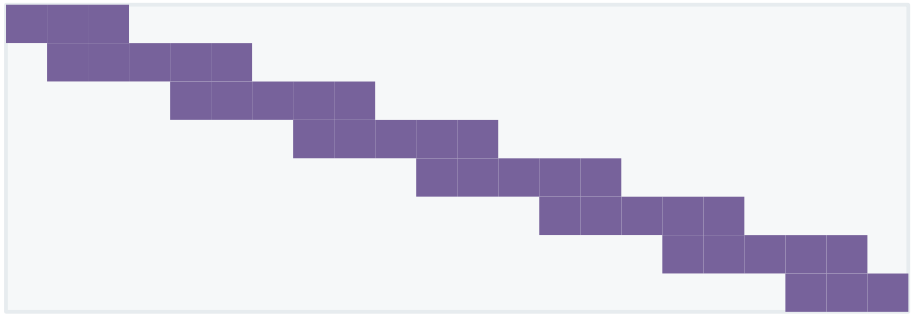
Numerical excerpts: first rows / columns. Full patterns above; zeros outside these excerpts are not implied.

# Transfer operators $P_1$ and $R_1$

$P_1$: 22 × 8. $R_1$: 8 × 22. Fixed algebraic transfer operators.



coarse correction to fine



fine residual to coarse: gnext = R d

| i / j | 1 | 2 | 3 | 4 |
|---|---|---|---|---|
| 1 | 0.306 | 0.000 | 0.000 | 0.000 |
| 2 | 0.408 | 0.181 | 0.000 | 0.000 |
| 3 | 0.313 | 0.376 | 0.000 | 0.000 |
| 4 | 0.000 | 0.577 | 0.000 | 0.000 |
| 5 | 0.000 | 0.395 | 0.193 | 0.000 |

| i / j | 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|---|
| 1 | 0.306 | 0.408 | 0.313 | 0.000 | 0.000 |
| 2 | 0.000 | 0.181 | 0.376 | 0.577 | 0.395 |
| 3 | 0.000 | 0.000 | 0.000 | 0.000 | 0.193 |
| 4 | 0.000 | 0.000 | 0.000 | 0.000 | 0.000 |

Numerical excerpts: first rows / columns. Full patterns above; zeros outside these excerpts are not implied.

# Transfer operators $P_2$ and $R_2$

Full final transfer pair. $R_2 = P_2^T$. Both operations run on the CPU.

| i / j | 1 | 2 | 3 |
|---|---|---|---|
| 1 | 0.2263 | 0.0000 | 0.0000 |
| 2 | 0.3424 | 0.2010 | 0.0000 |
| 3 | 0.3205 | 0.3753 | 0.0000 |
| 4 | 0.0000 | 0.5773 | 0.0000 |
| 5 | 0.0000 | 0.3754 | 0.2405 |
| 6 | 0.0000 | 0.2020 | 0.4377 |
| 7 | 0.0000 | 0.0000 | 0.4885 |
| 8 | 0.0000 | 0.0000 | 0.1654 |

| i / j | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 |
|---|---|---|---|---|---|---|---|---|
| 1 | 0.2263 | 0.3424 | 0.3205 | 0.0000 | 0.0000 | 0.0000 | 0.0000 | 0.0000 |
| 2 | 0.0000 | 0.2010 | 0.3753 | 0.5773 | 0.3754 | 0.2020 | 0.0000 | 0.0000 |
| 3 | 0.0000 | 0.0000 | 0.0000 | 0.0000 | 0.2405 | 0.4377 | 0.4885 | 0.1654 |

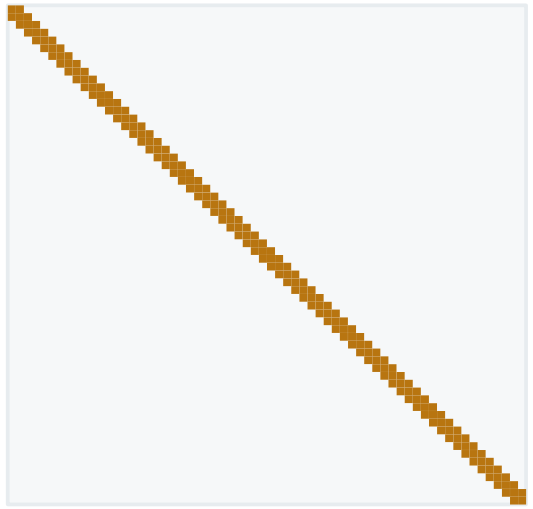
$P_2e_3$ has 8 entries.

$R_2d_2$ has 3 entries.

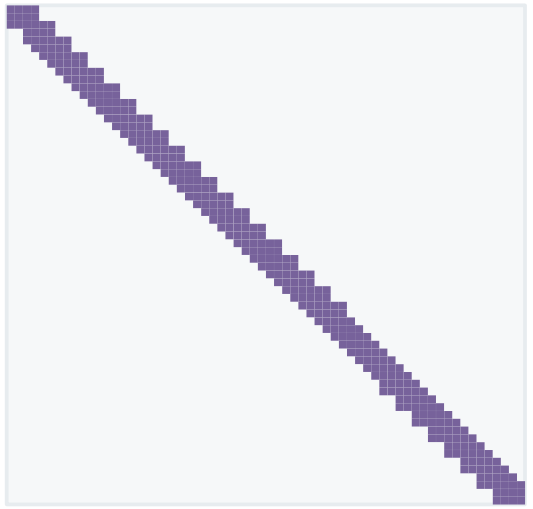
# The 2D experiment uses four fixed sparse matrices

main.py hierarchy from the measured run; distinct from the browser matrices

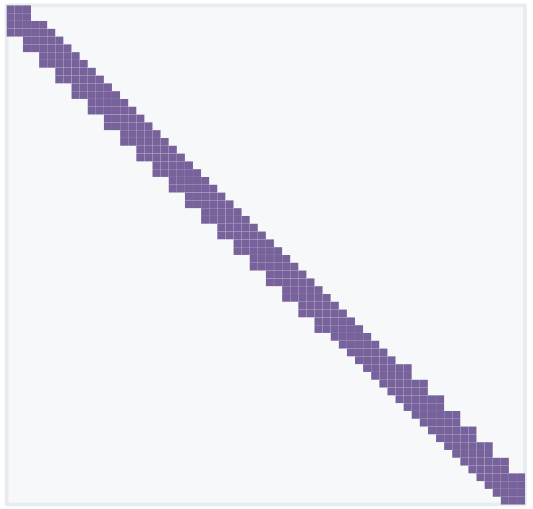
**A: 4096 × 4096**
**20,224 nonzeros**



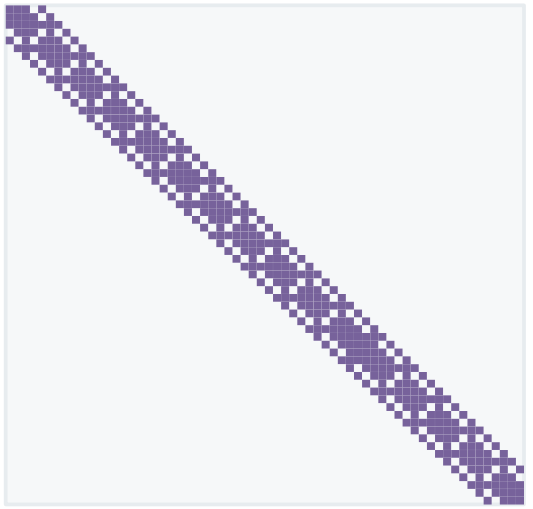
**P: 4096 × 704**
**10,116 nonzeros**



**R: 704 × 4096**
**10,116 nonzeros**



**Ac: 704 × 704**
**6,096 nonzeros**



$$A = I_{64} \otimes T_{64} + T_{64} \otimes I_{64} \qquad Ac = RAP \qquad R = P^T$$

$T_{64}$ = tridiag(−1, 2, −1). Patterns use 64 × 64 occupancy bins. Coarse solve: CPU.

# The noisy smoother has a specific software model

main.py: input quantization + independent Gaussian output noise

$$a = ||v||_\infty$$
$$q = \mathrm{round}(127\ v / a) / 127$$
$$\tilde{y} = 4a\ [\ (Aq)/4 + \xi\ ], \quad \xi_i \sim N(0, 0.045^2)$$

If a = 0, return zero.

Four pre-sweeps begin with e = ωg, then use three analog MVMs.

No post-smoothing in the hybrid configuration.

Not modeled: matrix quantization, optical averaging, device drift, saturation, conversion timing, or energy.

# Settings and counter definitions

| Setting | main.py hybrid | Browser |
|---|---|---|
| Outer target / cap | $10^{-10}$ / 100 restarts | $10^{-12}$ / 10 corrections |
| Inner CG target / cap | $10^{-12}$ / 5 steps | $10^{-2}$ / 20 steps |
| Smoothing | 4 pre / 0 post | 3 pre / 3 post |
| Digital counter | Fine A wrapper calls | $A\ell$, $P\ell$ and $R\ell$ calls |
| Zero-input smoothing | First product skipped | Explicitly counted |
| Coarse solve | 704 × 704 sparse solve | 3 × 3 direct solve |
| Convergence evidence | Independent true residual | True residual + terminal status |

# Convergence theory and observed results

Classical PCG assumes a fixed, linear, SPD preconditioner.

Noise and quantization make this V-cycle variable.

$$A(x^* - x) = r$$

$$x^* - x = A^{-1}r$$

A small residual measures satisfaction of the discrete equations.

Solution error also depends on conditioning. PDE error also depends on discretization.

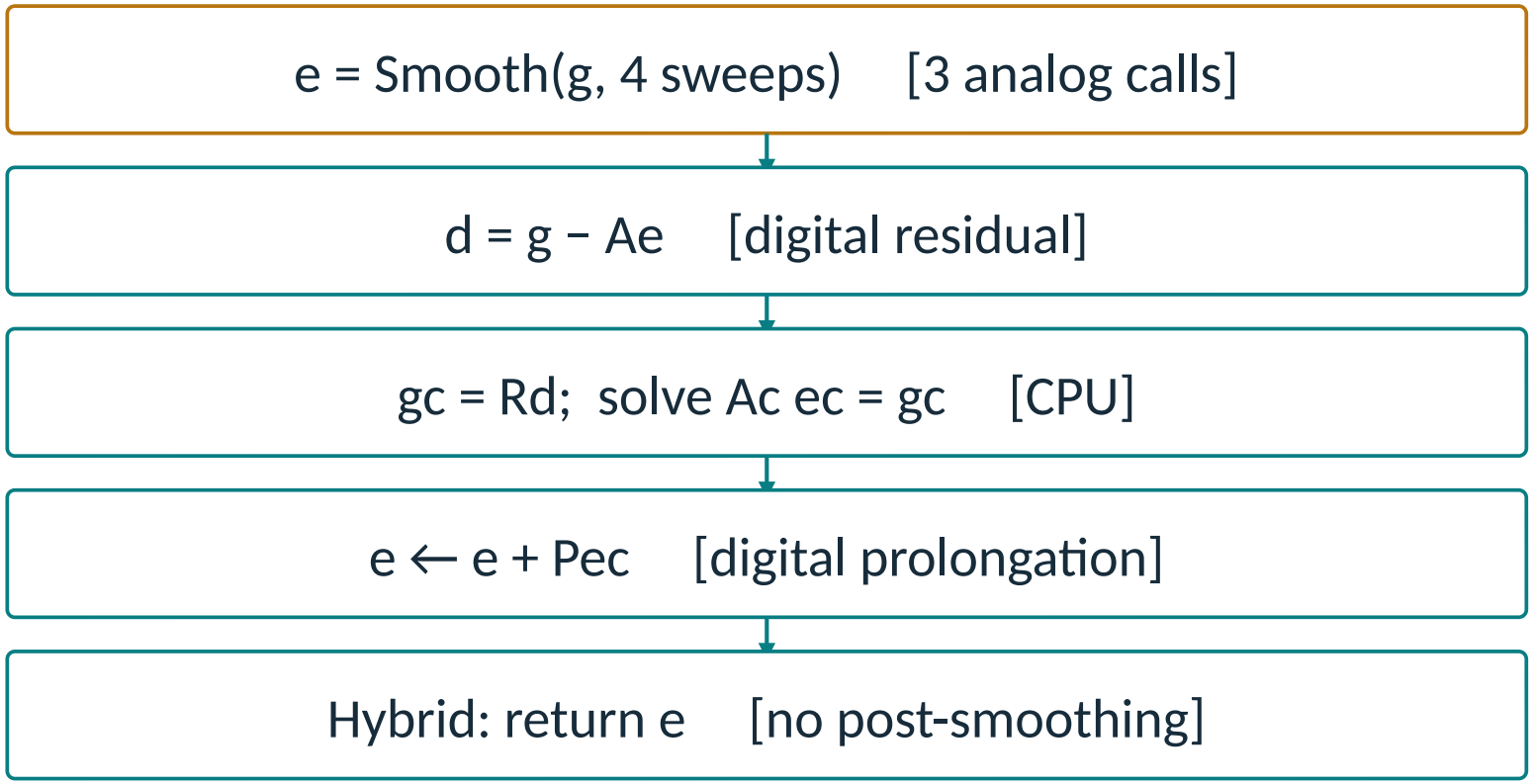
# Related work: Zhu’s coherent processor

| Aspect | Lee-inspired project | Zhu related work |
|---|---|---|
| Numerical emphasis | Multigrid inside Krylov + residual correction | Fixed-point iteration with error management |
| Optical assignment | Smoothing matrix–vector products | Coherent signed products |
| Electronics | Residuals, vectors, transfers, control | Memory, reconfiguration, control |
| Project status | Numerical emulation and ideal demonstrator | Architecture reference; not reproduced |

Common principle: approximate optical linear operations cooperate with digital state and correction.

# The actual two-level cycle in main.py

4096 → 704. Hybrid: four pre-sweeps, no post-smoothing.




Digital baseline:
2 pre + 2 post

Plain / digital CG:
solve Ax = b

Hybrid only:
short correction solves
with outer restarts

The coarse solver runs on a 704 × 704 matrix each cycle. Its cost is absent from the MVM counters.

# Setup constructs operators before iteration starts




| Browser level | Stored relaxation weight | Use |
|---|---|---|
| 0 | 5.92061673e-05 | Richardson smoothing |
| 1 | 0.000534826731 | Richardson smoothing |
| 2 | 0.0049999113 | Richardson smoothing |
| 3 | 0.0485813754 | Unused: direct solve |

main.py

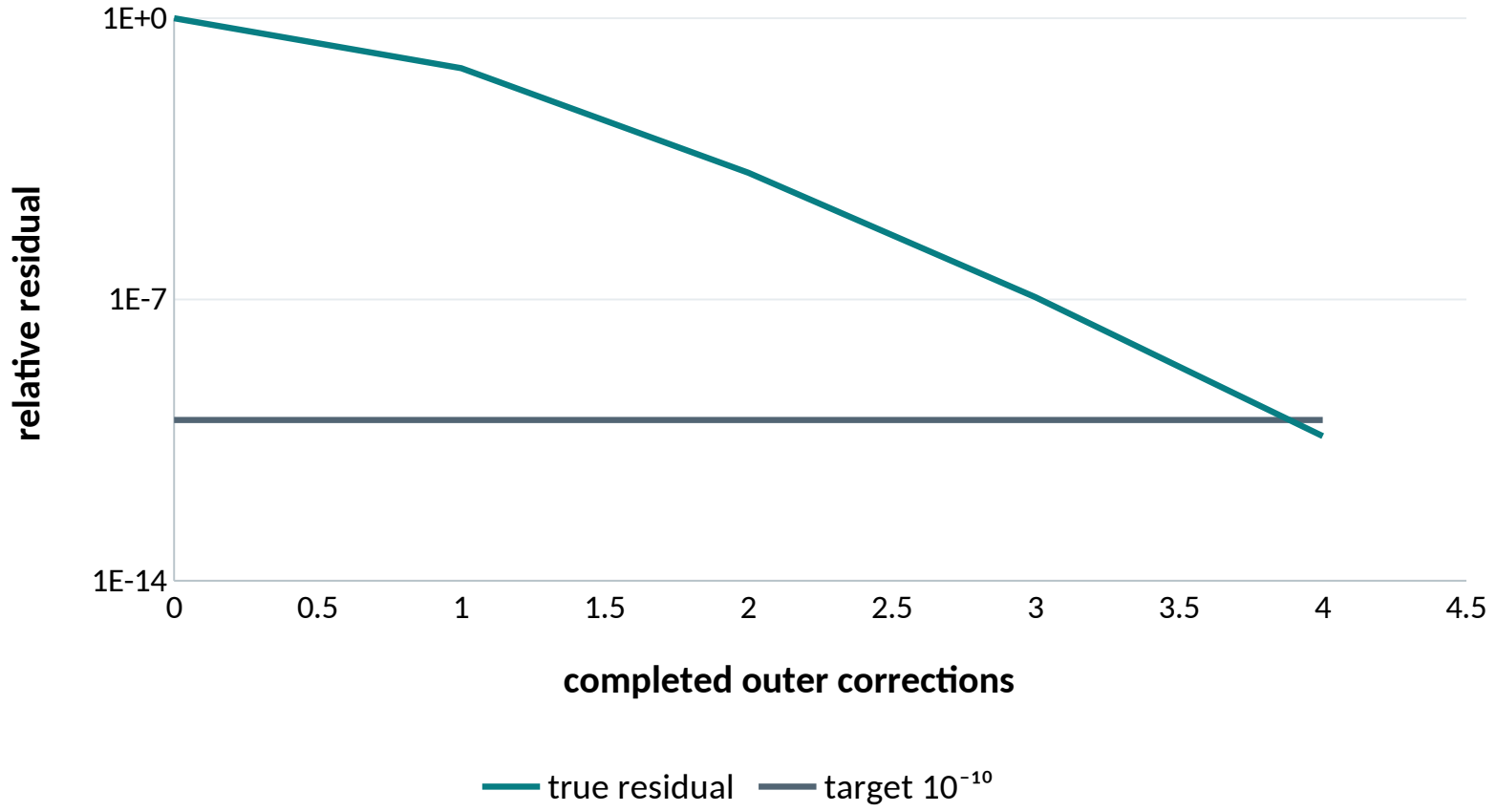
30 power iterations

ω = 0.12717828

During the solve: A, P, R, and ω stay fixed. Only solution, residual, and correction vectors change.

# True residuals contract across noisy restarts

main.py noisy hybrid. Each correction solve has at most five CG steps.




Four corrections

20 V-cycles

60 analog calls

Observed convergence for this seeded run; robustness across noise realizations remains to be measured.

Appendix · main.py: original solver, observed by presentation/build/collect_python.py; independently recomputed residual history

# Sources and reproducibility

Lee et al. (2025), Integrated photonic multigrid solver for partial differential equations
arXiv:2511.01005

Zhu et al. (2024), Coherent General-Purpose Photonic Matrix Processor
ACS Photonics 11, 1189–1196. DOI: 10.1021/acsphotonics.3c01694

Repository evidence: main.py and web/index.html

Browser: all three initial guesses reproduced.

2D: original noise generator seed 0; PyAMG setup fixed with NumPy seed 0.

Run environment: numpy 2.5.2, scipy 1.18.1, pyamg 5.3.0

Source SHA-256: 1dd6d0bcfa05058dbdf3730a…